Un moteur d'inférence pour eForth

version 1.1 - 4. novembre 2026



Marc PETREMANN

Table des matières

Introduction **3**

L'environnement de développement **4**
- Le fichier INFERENCE.FS 4
- Le projet main.fs 4
- Les autres fichiers 5

Définir des faits **6**
- Créer un fait 6
  - Autodocumenter un fait 7
- Création et gestion du vocabulaire facts 8
  - Listage des faits 8
  - Remise à zéro des faits 9
  - Affectation d'une valeur à un fait 9

Définir des règles **11**
- Créer une règle 11
  - Autodocumentation des règles 13
- Création et gestion du vocabulaire rules 14
  - Remise à zéro des règles 14

Améliorations générales du code **16**
- Utiliser la récursivité 16
- Un code multilingue 18
  - Application à la déclaration de faits et règles 19

Ressources **20**
- Github 20
  - rules-engine 20
  - simple-forth-expert 20
- Web 20
  - DELTA: An Expert System for Diesel Electric Locomotive Repair 20
  - FORPS: a FORTH-based production system and its application 20

Introduction

Un **moteur d'inférence** est le composant central d'un système expert ou d'une intelligence artificielle symbolique, responsable de la prise de décision et du raisonnement automatisé. Il fonctionne en combinant une **base de connaissances** (constituée de faits et de règles de production) avec des algorithmes de déduction pour en déduire de nouvelles informations.

Il utilise principalement deux stratégies de raisonnement :

- **Le chaînage avant**, qui part des faits disponibles pour arriver à une conclusion.
- **Le chaînage arrière**, qui part d'une hypothèse cible pour remonter aux faits nécessaires qui la valident.

En somme, il simule la logique humaine pour résoudre des problèmes complexes sans nécessiter de programmation procédurale rigide.

L’environnement de développement

Pour le moteur d’inférence, nous utiliserons la version eForth Windows. C’est une version FORTH utilisable pour Windows, disponible ici :

https://github.com/MPETREMANN11/eForth-Windows/tree/main/__program

La documentation de cette version est accessible ici :

https://github.com/MPETREMANN11/eForth-Windows/tree/main/__documentation

eForth ne nécessite aucune installation.

Téléchargez simplement le fichier **uEf64-7.0.7.21.exe** dans un répertoire, à l’emplacement de votre choix, nommé par exemple **eForth**.

Ouvrez ce répertoire **eForth**. Créez un nouveau dossier **INFERENCE**. Téléchargez les fichiers nécessaires depuis le dépôt logiciel. Vous devriez avoir :

```
📁 eForth
    uEf64-7.0.7.21.exe
    INFERENCE.fs
    📁 INFERENCE
        config.fs
        facts.fs
        inference-engine.fs
        main.fs
        rules.fs
        tests.fs
```

Le fichier INFERENCE.FS

C’est le fichier qui permet de se connecter au reste du projet. Contenu :

```
\ load INFERENCE
s" INFERENCE/main.fs" included
```

A chaque fois que vous voudrez compiler le code et utiliser le moteur d’inférence, il suffira de lancer **eForth** en cliquant sur le fichier **uEf64-7.0.7.21.exe**. Une fois eForth démarré, tapez :

```
include INFERENCE.fs
```

Le projet main.fs

L’ensemble du projet est compilé depuis **main.fs**. Exemple de contenu :

```
include config.fs               \ load config
include inference-engine.fs     \ load inference engine
```

Vous pouvez rajouter le chargement d’autres fichiers. Par exemple, pour compiler **tests.fs** :

```
include config.fs               \ load config
include inference-engine.fs     \ load inference engine
include tests.fs
```

Les autres fichiers

Ils sont chargés depuis **main.fs**. Le projet est découpé en fonctionnalités utilisables selon l'évolution du projet. La liste et le contenu des fichiers sources peut évoluer sans préavis et ne plus être ceux décrits dans ce document.

L'ordre de chargement des fichiers est essentiel. Ainsi, on ne peut pas charger le contenu de **facts.fs** avant d'avoir traité le contenu de **inference-engine.fs**.

Définir des faits

Vous pensez maîtriser la logique ? C'est parfait. Alors vous savez manipuler les opérateurs logiques **ET**, **OU**, **NON**… Alors voyons un cas pratique avec un exemple tout à fait trivial :

- la température est très froide
- il pleut

Nous avons deux faits. On peut déduire que s'il pleut et qu'il fait très froid, le risque de gel est réel.

On peut coder ceci en FORTH :

```
 0 constant FALSE
-1 constant TRUE

VARIABLE fait-pluie
VARIABLE fait-froid
VARIABLE fait-gel

: RAZ-FAITS ( -- )
    FALSE fait-pluie !
    FALSE fait-froid !
    FALSE fait-gel ! ;
```

Nos variables **fait-pluie** et **fait-gel** sont les drapeaux à tester, avec les valeurs 0 pour faux, -1 pour vrai. Sur ces deux faits, nous pouvons appliquer ces règles :

```
\ Définition des règles comme des mots Forth standards
: cond-regle1 ( -- flag )
    fait-pluie @  fait-froid @  AND ;
: act-regle1  ( fl -- )
    cond-regle1 if
        TRUE fait-gel !
    then ;
```

A ce stade, nous ne pouvons pas encore parler d'inférence. Et une seule règle, ici **act-regle1** ne justifie pas la création d'un moteur d'inférence.

Créer un fait

C'est le premier pas vers un moteur d'inférence. Un fait est une variable qui contient un état logique ou une valeur numérique. Pour définir un fait :

```
: FACT: ( n|fl comp: -- <word> | exec: -- n )
    value
  ;
```

Ce qui nous permet de réécrire les variables ainsi :

```
FALSE FACT: fait-pluie
FALSE FACT: fait-froid
FALSE FACT: fait-gel
```

Le mot **FACT:** crée un fait qui est simplement une valeur. L'exécution d'un fait empile sa valeur :

```
21 FACT: temperature
temperature   \ empile 21
```

Autodocumenter un fait

L'idée est de greffer une chaîne de caractères qui informe sur la nature du contenu du fait déclaré. EForth sait gérer les chaînes terminées par zéro :

```
z" Température extérieure" z>s type  \ affiche texte:  Température extérieure
```

Pour greffer ce type de chaîne à un fait, on va simplement compiler l'adresse de cette chaîne après la valeur :

```
: FACT: ( n|fl comp: -- <word> | exec: -- n )
    value                   \ create value
    ,                       \ compile pointer to zstring
  ;
```

Et s'utilise comme ceci :

```
z" Température extérieure" 21 FACT: ext-temp
```

L'exécution de **ext-temp** empile la valeur 21.

Pour récupérer la chaîne de caractères compilée après cette valeur, il faut explorer le champ de données de **ext-temp** :

```
' ext-temp >body cell + @ z>s cr type
\ affiche: Température extérieure
```

Pour faciliter l'accès aux données dans le champ de données d'un fait créé par **FACT:** , on définit cette structure :

```
structures
struct __FACTS
    ptr field ->value           \ fact value
    ptr field ->factZstring     \ point to fact zstring
forth
```

Pour connaître le statut d'un fait, on va définir ce mot :

```
\ display fact zstring and value in fact
\ example: ' <fact> explain-fact
: explain-fact { xtAddr -- }
    xtAddr >name type
    xtAddr >body ->factZstring @ z>s cr  4 spaces type ."  = "
    xtAddr >body ->value @ { value }
    value 0= if
        ." FALSE"
    then
    value -1 = if
        ." TRUE"
    then
    value 0=  value -1 = or invert if
```

```
        value .
    then
    cr cr
  ;
```

Et s'utilise ainsi :

```
' ext-temp explain-fact   \ display:
\ ext-temp
\     Température = 21
```

Création et gestion du vocabulaire facts

En l'état, tous les faits sont définis dans le vocabulaire **`FORTH`**. Si on mixe des définitions classiques, des règles, des faits, l'exécution de **`vlist`** va restituer une liste de mots difficile à gérer. L'idée serait donc de définir tous les faits et seulement ces faits dans un vocabulaire spécifique :

```
vocabulary facts    \ new facts defined in this vocabulary
```

Pour basculer une création de faits dans ce vocabulaire, on définit des sélecteurs de vocabulaires :

```
: in-facts ( -- )
    facts definitions
  ;

: in-forth ( -- )
    only
    forth definitions
  ;
```

Ces sélecteurs sont incorporés à la définition de **`FACTS:`** :

```
\ create a new fact
: FACT: ( n|fl comp: -- <word> | exec: -- n )
    in-facts
    value                   \ create value
    ,                       \ compile pointer to zstring
    in-forth
  ;
```

Définissons ces faits :

```
z" Pleut-il" FALSE   FACT: fait-pluie
z" Température extérieur" -4   FACT: ext-temp
z" Gèle-t-il" FALSE  FACT: fait-gel
```

Ces faits seront définis dans le vocabulaire **`facts`** et sont visibles :

```
facts vlist    \ affiche: fait-gel ext-temp fait-pluie
```

Listage des faits

L'idée sous-jacente de gestion des faits dans le vocabulaire **`facts`**, c'est de permettre ensuite le listage détaillé. Pour ce faire, on définit ce mot :

```
forth definitions also internals
: list-facts ( -- )
    facts context @ @
    begin  dup nonvoc?
    while  dup explain-fact cr cr >link
    repeat
    drop
    forth
  ;
only
```

L'exécution de `list-facts` sélectionne le vocabulaire `facts` et récupère le code exécutable de chaque fait. Ce code exécutable utilise `explain-fact`. Résultat :

```
list-facts    \ display:
\ fait-gel
\     Gèle-t-il = FALSE
\
\ ext-temp
\     Température = -4
\
\ fait-pluie
\     Pleut-il = FALSE
```

Cette liste de faits affiche les faits et sur la ligne suivante l'auto-documentation et la valeur du fait. Le mot `list-facts` permet donc de tracer tous les faits traités par le moteur d'inférence.

Remise à zéro des faits

La manière dont les faits sont listés est exploitée pour remettre à `FALSE` tous les faits :

```
\ reset fact value to FALSE
: (reset-fact) ( xt -- )
    >body FALSE swap !
  ;

\ reset values of all facts defined in facts vocabulary
forth definitions also internals
: reset-facts ( -- )
    facts context @ @
    begin  dup nonvoc?
    while  dup (reset-fact) >link
    repeat
    drop cr ." All facts set to FALSE"
    forth
  ;
only
```

Le mot `reset-facts` analyse chaque fait déclaré dans le vocabulaire `facts` et remet sa valeur à `FALSE`.

Affectation d'une valeur à un fait

Dans le fichier **vocForthExtensions.fs** se trouve le mot `val-input` :

```
\ wait a value and push it on stack
```

```
: val-input ( -- n )
    PAD 80 ACCEPT
    PAD SWAP
    EVALUATE
  ;
```

Le contenu du fichier **vocForthExtensions.fs** est compilé par **main.fs** avant la compilation du moteur d'inférence. Le mot `val-input` attend la saisie d'une valeur numérique depuis le clavier et empile sa valeur. Utilisation :

```
facts
cr ." Température actuelle: " val-input to ext-temp  \ saisir 27
ext-temp .   \ affiche 27
```

A noter que vous pouvez également saisir `TRUE` ou `FALSE`, car `val-input` effectue une évaluation de la valeur saisie. Si cette valeur est une constante ou valeur du vocabulaire FORTH, ce mot sera exécuté et sa valeur déposée sur la pile de données.

Définir des règles

Les règles traitent les faits en deux étapes :

1. vérifie un ou plusieurs faits ;
2. traite un ou plusieurs faits si la règle est vérifiée

Exemple :

```
FALSE value F1
FALSE value F2
FALSE value F3
: R1  ( -- )   \ our first rule
    F1 F2 and if
        TRUE to F3
    then ;
```

Rien de bien compliqué. Factorisons la partie *condition* et la partie *action* :

```
: condition-R1 ( -- fl )
    F1 F2 and ;
: action-R1 ( -- )
    TRUE to F3 ;
: R1  ( -- )   \ our first rule
    condition-R1 if
        action-R1
    then ;
```

Créer une règle

L'idée est de définir un mot (RULE:) qui gère une condition et une action. Code de ce mot :

```
structures
struct __RULES
    ptr field ->condition   \ Rule condition field
    ptr field ->action      \ field of the rule's action part
    ptr field ->execFlag    \ rule execution flag field, set to TRUE if rule
executed
forth

\ Executes the condition
: exec-condition ( addr -- fl )
    ->condition @ execute
  ;

\ execute the action
: exec-action ( addr -- )
    ->action @ execute
  ;

\ Mark rule as triggered
\ Indicate that a rule triggered in this cycle
: set-flag { addr -- }
    TRUE addr ->execFlag !
    TRUE to MODIFIED_RULE
  ;\ create a new rule
```

```
: (RULE:)
    create
        swap , ,    \ store xt condition and xt action
        0 ,         \ store initial flag, value FALSE
    does>
        { addr }
        \ Execute rule if has not yet been triggered in the cycle
        addr ->execFlag @ 0= if
            addr ->condition @ execute if
                addr ->action @ execute
                addr set-flag
            then
        then
  ;
```

Dans le mot **(RULE:)**, la partie qui suit **CREATE** :

- on stocke le code d'exécution de la condition,
- on stocke ensuite le code d'exécution de la partie action,
- pour terminer, on stocke un drapeau qui est basculé à **TRUE** si la règle est invoquée.

Exemple de définition d'une règle :

```
:NONAME loanAmount 0 >  loanTerms 0 > and interestRate 0 > and ;
:NONAME loanAmount loanTerms interestRate MENSUALITE-EXACTE to monthlyInstallment ;
RULE: calcMonthlyInstallment
```

La séquence **:NONAME <definition> ;** dépose sur la pile le code d'exécution du code FORTH déposé après **:NONAME**.

Pour rendre la création d'une règle plus lisible, on définit ces deux mots :

```
\ define condition of a rule
: :CONDITION ( -- fl )
    facts
    :NONAME
  ;

\ define action for a rule
: :ACTION ( -- )
    facts
    :NONAME
  ;
```

Ce qui permet de réécrire la définition de notre règle :

```
:CONDITION loanAmount 0 >  loanTerms 0 > and interestRate 0 > and ;
:ACTION    loanAmount loanTerms interestRate MENSUALITE-EXACTE to
monthlyInstallment ;
(RULE:) calcMonthlyInstallment
```

Notre règle **calcMonthlyInstallment** exécute séquentiellement :

- analyse le drapeau d'exécution initialement à zéro. Si le drapeau est nul, c'est que la règle n'a pas encore été traitée ;

- dans ce cas, exécute le code de la condition. Ce code doit toujours délivrer un drapeau **TRUE** ou **FALSE**.
- Si le drapeau de la condition est à **TRUE**, exécute le code de l'action.

Dans notre exemple, on teste trois faits **loanAmount**, **loanTerms** et **interestRate**. Si ces faits sont différents de zéro, la condition est vraie. Dans ce cas, l'action est exécutée. Ici, on calcule la mensualité d'un crédit et on affecte le résultat au fait **monthlyInstallment**. Puis le drapeau d'exécution de la règle est basculé à **TRUE**. Quand ce drapeau est vrai, la règle ne peut plus être exécutée.

Un autre drapeau **MODIFIED_RULE** est basculé à vrai. C'est un drapeau global. A chaque inférence, il est mis à FALSE. Si une seule règle est exécutée, il est positionné à **TRUE**. Nous verrons plus loin comment ce drapeau est géré par le moteur d'inférence.

Autodocumentation des règles

Comme pour les faits, il est facile d'autodocumenter une règle :

```
z" Calculated monthly loan payment"
:CONDITION loanAmount 0 >  loanTerms 0 > and interestRate 0 > and ;
:ACTION    loanAmount loanTerms interestRate EXACT-MONTHLY-PAYMENT to
monthlyInstallment ;
RULE: calcMonthlyInstallment
```

L'autodocumentation d'une règle est intégrée après CREATE dans la définition de la règle, extrait du code :

```
\ create a new rule
: (RULE:)
    create
        swap , ,    \ store xt condition and xt action
        0 ,         \ store initial flag, value FALSE
        ,           \ store auto-documentation string pointer
    does>
        \ DOES code here
  ;
```

Pour afficher cette autodocumentation, on utilisera le mot explain-rule, précédé du code d'exécution de la règle à afficher :

```
\ display rule zstring and name in rules
\ example: ' <rule> explain-rule
: explain-rule { xtAddr -- }
    xtAddr >name type
    xtAddr >body ->ruleZstring @ z>s cr  4 spaces type ."  = "
    xtAddr >body ->execFlag @
    0= if
        09 bg ."  KO " 0 bg     \ red background
    else
        10 bg ."  OK " 0 bg     \ green background
    then
    cr cr
  ;
```

Exemple :

```
' calcMonthlyInstallment explain-rule
```

Affiche :

```
calcMonthlyInstallment
    Mensualité calculée du crédit = KO
```

Ici, le résultat **KO** indique que la règle n'a pas encore été exécutée.

Création et gestion du vocabulaire rules

Tout comme nous avons défini le vocabulaire **facts**, on définit et exploite le vocabulaire **rules** :

```
vocabulary rules    \ new rules defined in this vocabulary
```

Pour basculer une création de règles dans ce vocabulaire, on définit son sélecteur de vocabulaire:

```
: in-rules ( -- )
    rules definitions
  ;
```

Ce sélecteur **in-rules** est exploité ainsi :

```
\ define new rule in rules vocabulary
: RULE: ( -- )
    in-rules
    (RULE:)
    in-forth
  ;
```

Reprenons notre exemple de création de règle :

```
:CONDITION loanAmount 0 >  loanTerms 0 > and interestRate 0 > and ;
:ACTION    loanAmount loanTerms interestRate MENSUALITE-EXACTE to
monthlyInstallment ;
RULE: calcMonthlyInstallment
```

Maintenant notre règle **calcMonthlyInstallment** est définie dans le vocabulaire **rules**.

Pour retrouver toutes les règles définies dans le vocabulaire **rules**, on exécutera ceci :

```
rules vlist   \ affiche:
hightDebtTest evalHightIncome calcMonthlyInstallment getLoanAmount getMonthIncome
```

A ce stade, on pourrait également définir **list-rules** de manière similaire à **list-facts**. Nous verrons une amélioration significative de leur définition dans le chapitre suivant.

Remise à zéro des règles

Voici le code qui remet à **FALSE** tous les drapeaux d'exécution des règles :

```
\ Resets the execution flag for all rules
forth definitions also internals
: reset-rules ( -- )
```

```
    rules context @ @
    begin dup nonvoc?
    while
        FALSE over >body ->execFlag !  \ set ->execFlag à FALSE
        >link
    repeat
    drop
  ;
only
```

Le mot **reset-rules** analyse chaque règle déclarée dans le vocabulaire **rules** et remet sa valeur à **FALSE**.

Améliorations générales du code

Avant de poursuivre sur le fonctionnement même du moteur d'inférence, il est bon d'expliquer certaines améliorations apportées sur le code et les fonctionnalités générales.

Dans le chapitre *Définir les faits*, le mot `list-facts` liste les faits. L'inconvénient de son fonctionnement, il liste les faits dans l'ordre inverse de leur déclaration. Le fait le plus ancien sera affiché en dernier, alors qu'il a été écrit en premier dans le code source FORTH.

Par exemple, si on définit F1, F2, F3… Fn, on verra les faits listés dans l'ordre Fn… F3 F2 F1. Cet ordre n'est pas satisfaisant, car la déclaration des faits obéit à un ordre logique.

Comment respecter l'ordre de déclaration ?

Utiliser la récursivité

C'est le mot qui fait peur. Car une récursivité mal maîtrisée mène à la catastrophe. Pour ceux qui ne connaissent pas la récursivité, voici comment ça fonctionne :

```
: listNumbers ( n -- )
    dup 1-  ?dup 0 > if
      recurse
    then
  ;
```

L'exécution de `10 listNumbers` dépose tous les entiers de 1 à 10 sur la pile de données.

Dans la définition de `listNumbers` le mot `recurse` va appeler le mot `listNumbers`. C'est comme si on avait écrit :

```
: listNumbers ( n -- )
    dup 1-  ?dup 0 > if
      listNumbers
    then
  ;
```

Dans une récursivité, il faut obligatoirement tester une condition qui limite la profondeur de cette récursivité. La récursivité empile une adresse de retour qui est le code situé juste après le mot `recurse`. Ici, la condition d'arrêt de récursivité est le test `0 >`. Résultat de `listNumbers` :

```
--> 10 listnumbers
 ok
10 9 8 7 6 5 4 3 2 1 -->
```

Pour notre exemple, si on rajoute une action après `recurse`, cette action va traiter les valeurs autant de fois qu'il y a d'actions en attente sur la pile de retour. Ici on va simplement rajouter la multiplication `*` :

```
: factorial ( n – factn )
    dup 1-  ?dup 0 > if
      recurse
```

```
      *
    then
  ;
```

Test : `6 factorial` empile 720 qui est la factorielle de 6.

Venons-en maintenant à la manière d'exploiter cette récursivité pour traiter les faits dans l'ordre de déclaration. Si vous faites `facts vlist`, vous aurez la liste de tous les faits déclarés dans l'ordre inverse de définition dans le code source. L'idée est de copier le fonctionnement de `vlist`, donc d'extraire chaque fait en empilant son code d'exécution sur la pile de données au lieu d'afficher ce mot. Si on empile le code xt de chaque mot du vocabulaire `facts`, arrivé au dernier mot, qui est celui compilé en premier, on se retrouve avec le code xt de ce mot sur le sommet de la pile.

Par exemple, si on déclare F1 F2 F3, il faut empiler les codes xt de F3, puis F2 et F1.

```
\ Explore facts vocabulary recursively and display facts
: (lr-fac) ( thread -- )
    dup nonvoc? if
        dup >link recurse
        explain-fact
    else
        drop
    then ;
```

Le mot `(lr-fac)` pour *list-recursiveley-facts*, est le squelette d'une définition améliorée de `list-facts`.

```
forth definitions also internals
\ Explore facts vocabulary recursively and display facts
: (lr-fac) ( thread -- )
    dup nonvoc? if
        dup >link recurse
        explain-fact
    else
        drop
    then ;

: .facts-banner ( -- )
    cr 14 bg space
    [FRANCAIS] ?\ z" LISTE DES FAITS:"
    [DEUTSCH]  ?\ z" LISTE DER FAKTEN:"
    [ENGLISH]  ?\ z" LIST OF FACTS:"
    space 0 bg cr cr
  ;

: list-facts ( -- )
    .facts-banner
    facts context @ @ (lr-fac) ;
only
```

Maintenant, `list-facts` affiche les faits et leurs valeurs dans l'ordre de compilation par FORTH.

Appliquons cette même récursivité à l'affichage des règles :

```
\ List all rules
forth definitions also internals
\ Explore rule vocabulary recursively and display rules
: (lr-rec) ( thread -- )
    dup nonvoc? if
        dup >link recurse
        explain-rule
    else
        drop
    then ;

: list-rules ( -- )
    .rules-banner
    rules context @ @ (lr-rec) ;
only
```

Un code multilingue

On vient d'en avoir un aperçu dans cette définition :

```
: .facts-banner ( -- )
    cr 14 bg space
    [FRANCAIS] ?\ z" LISTE DES FAITS:"
    [DEUTSCH]  ?\ z" LISTE DER FAKTEN:"
    [ENGLISH]  ?\ z" LIST OF FACTS:"
    space 0 bg cr cr
  ;
```

Cette définition exploite simplement le mot `?\` qui est une compilation conditionnelle de code. Code source de ce mot :

```
\ Conditionnal compilation
: ?\ ( fl -- )
    0= if
        nl parse drop drop
    then
; immediate
```

Ce mot doit être précédé d'une valeur booléenne. Pour ce faire, on a défini trois valeurs :

```
\ these values must be used in interpertation mode only
 0 value DEUTSCH
 0 value ENGLISH
 0 value FRANCAIS
```

Pour être utilisables dans le corps d'une définition, on crée leur alias d'exécution immédiate :

```
\ these words must be used in compilation mode only
: [DEUTSCH]
    DEUTSCH ;  immediate

: [ENGLISH]
    ENGLISH ;  immediate

: [FRANCAIS]
    FRANCAIS ;  immediate
```

Pour sélectionner une langue :

```
: in-english ( -- )
    -1 to ENGLISH
  ;

: en-francais ( -- )
    -1 to FRANCAIS
  ;

: auf-deutsch ( -- )
    -1 to DEUTSCH
  ;
```

Application à la déclaration de faits et règles

Voici comment déclarer un fait en exploitant cette option multilingue :

```
FRANCAIS ?\ z" Revenu mensuel net après impôts"
DEUTSCH  ?\ z" Monatliches Nettoeinkommen nach Steuern"
ENGLISH  ?\ z" Net monthly income after taxes"
FALSE FACT: monthIncome
```

Version sans option multilingue :

```
z" Net monthly income after taxes"
FALSE FACT: monthIncome
```

Le choix de la langue s'effectue au moment de compiler le code FORTH, dans **main.fs** par exemple :

```
false value EV_DIAG     \ facts and rules for electric car diagnostic
true  value FR_LOAN     \ facts and rules for financial loans

en-francais

FR_LOAN [IF]
    cr ."  - faits et règles accord de prêt"
    include FactsRulesLoan.fs
[THEN]

EV_DIAG [IF]
    cr ."  - faits et règles diagnostic véhicule électrique"
    include FactsRulesEVdiag_FR.fs
[THEN]
```

La compilation conditionnelle permet de compiler un code FORTH adapté à la langue de l'utilisateur final. Les options de sélection linguistiques peuvent être étendues sans souci à d'autres langues depuis le fichier **vocForthExtensions.fs**.

Ressources

Github

rules-engine

Petit système expert pour Swiftforth

https://github.com/Pfontvilanova/rules-engine

simple-forth-expert

Un système expert simple de type « oui/non », écrit en Forth. Conçu à l'origine pour déterminer quel variant du COVID-19 une personne pourrait avoir. Testé sur ForthWin.

https://github.com/dicpeynado/simple-forth-expert/tree/main

https://www.youtube.com/watch?v=W38Hy3JgnEA

Web

DELTA: An Expert System for Diesel Electric Locomotive Repair

Le département de recherche et développement de la société General Electric a appliqué la technologie des systèmes experts au problème du dépannage et de la réparation des locomotives diesel-électriques dans les ateliers de maintenance ferroviaire. Ce système expert utilise des règles de production ainsi qu'un moteur d'inférence capables de diagnostiquer divers problèmes sur la locomotive et de suggérer des procédures de réparation au personnel de maintenance. Un prototype a été développé en langage FORTH et fonctionne sur un ordinateur PDP 11/23 de Digital Equipment, sous le système d'exploitation RSX-11M.

https://www.researchgate.net/publication/235120711_DELTA_An_Expert_System_for_Diesel_Electric_Locomotive_Repair

https://vfxforth.com/flag/jfar/vol1/no1/article1.pdf

FORPS: a FORTH-based production system and its application

FORPS (FORTH-based Production System) est un système de production en intelligence artificielle, fondé sur des règles, conçu au sein de l’environnement de programmation FORTH.

https://vfxforth.com/flag/jfar/vol4/no2/article10.pdf